

# Deconvolution GUI

## Program Description and Numerical Methods

Version 0.103.2

<https://github.com/rkotynski/dekonwolucje>

This document describes the graphical workflow, numerical data model, convolution operators, implemented deconvolution methods, quality criteria, automatic parameter tuning, and software architecture of the **dekonwolucje** project. The software is distributed under the MIT License. Source-code comments and internal identifiers remain in English independently of the selected English or Polish GUI language.

August 2026

# Contents

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1 Purpose and scope</b>                                | <b>2</b> |
| 1.1 AI-assisted development disclosure . . . . .          | 2        |
| <b>2 Forward model and boundary conditions</b>            | <b>2</b> |
| 2.1 Zero-boundary linear convolution . . . . .            | 2        |
| 2.2 Circular FFT model . . . . .                          | 2        |
| <b>3 Committed calculation image and PSF</b>              | <b>2</b> |
| 3.1 Source arrays and reset . . . . .                     | 2        |
| 3.2 Image and PSF floors . . . . .                        | 3        |
| 3.3 Rectangular PSF selection and normalization . . . . . | 3        |
| 3.4 Automatic initial support . . . . .                   | 3        |
| <b>4 Graphical workflow</b>                               | <b>3</b> |
| 4.1 Tab 1: image and PSF . . . . .                        | 3        |
| 4.2 Tab 2: thresholds and calculation PSF . . . . .       | 3        |
| 4.3 Tab 3: algorithm . . . . .                            | 4        |
| 4.4 Tab 4: results . . . . .                              | 4        |
| <b>5 Wiener filtering</b>                                 | <b>4</b> |
| <b>6 Richardson–Lucy family</b>                           | <b>4</b> |
| 6.1 Classical Richardson–Lucy . . . . .                   | 4        |
| 6.2 Richardson–Lucy–Wiener . . . . .                      | 4        |
| 6.3 Richardson–Lucy–Rosen . . . . .                       | 5        |
| <b>7 Landweber, Kaczmarz, and Adam methods</b>            | <b>5</b> |
| 7.1 Landweber . . . . .                                   | 5        |
| 7.2 Block Kaczmarz . . . . .                              | 5        |
| 7.3 PyTorch Adam TV-MAP . . . . .                         | 5        |
| <b>8 Blind deconvolution</b>                              | <b>5</b> |
| 8.1 Blind Richardson–Lucy . . . . .                       | 5        |
| 8.2 Blind Adam TV-MAP . . . . .                           | 5        |
| <b>9 Metrics and best-iteration selection</b>             | <b>6</b> |
| <b>10 Auto tuning and cancellation</b>                    | <b>6</b> |
| <b>11 Batch processing and CUDA</b>                       | <b>6</b> |
| <b>12 Software architecture</b>                           | <b>6</b> |
| <b>13 Algorithm registry</b>                              | <b>7</b> |

|                                         |   |
|-----------------------------------------|---|
| 14 Installation, testing, and extension | 7 |
| 15 Limitations and interpretation       | 7 |
| 16 License and project information      | 8 |

# 1 Purpose and scope

The program is a desktop research application for restoration of grayscale images blurred by a point-spread function (PSF) and contaminated by noise. It supports non-blind deconvolution with a supplied PSF and blind deconvolution in which both the latent image and the PSF are updated. The GUI separates source-data preparation, committed PSF selection, algorithm parameters, and result assessment.

The same committed calculation image and calculation PSF are used by all algorithms, synthetic degradation, reblur metrics, and blind-PSF initialization. NumPy/SciPy methods and PyTorch variants are available. PyTorch uses `float32` by default and can use CUDA when requested and available.

## 1.1 AI-assisted development disclosure

Parts of the software and documentation were prepared with assistance from tools based on large language models (LLMs). Their suggestions were incorporated as part of the development process. This assistance does not replace independent verification: numerical methods, implementation details, parameter choices, and reconstructed results should be validated for the intended scientific application.

# 2 Forward model and boundary conditions

Let  $f$  be the unknown image,  $h$  the PSF,  $g$  the measured image, and  $n$  additive noise. The discrete model is

$$g = \mathcal{H}f + n. \quad (1)$$

The project deliberately distinguishes two operators.

## 2.1 Zero-boundary linear convolution

Most iterative methods and synthetic degradation use linear convolution with zero values outside the image, followed by the exact centered *same* crop. The full FFT canvas has size

$$(M + M_h - 1) \times (N + N_h - 1). \quad (2)$$

The corresponding adjoint  $\mathcal{H}_{\text{lin}}^*$  is the transpose of this exact cropped operator, including the asymmetric convention needed for even kernels.

## 2.2 Circular FFT model

The closed-form Wiener inverse uses circular convolution because the operator is diagonal in the discrete Fourier basis. With

$$G = \mathcal{F}\{g\}, \quad H = \mathcal{F}\{h_0\}, \quad (3)$$

where the centered PSF is embedded on the image grid and shifted to the FFT origin,

$$\mathcal{H}_{\text{circ}}f = \text{Re} \left\{ \mathcal{F}^{-1} [H\mathcal{F}\{f\}] \right\}. \quad (4)$$

Linear and circular boundary models are not identical. A Wiener result can differ near the borders from an iterative result even when both use the same numerical PSF.

# 3 Committed calculation image and PSF

## 3.1 Source arrays and reset

The program retains the image and PSF loaded from disk or generated in the GUI as source arrays. **Reset thresholds / PSF selection** restores these sources. In parallel it stores one committed pair: the thresholded calculation image and the thresholded, cropped, nonnegative, unit-sum calculation PSF.

### 3.2 Image and PSF floors

For nonnegative data  $x$  with maximum  $m$  and floor  $T$ , the applied mapping is

$$x_T = \begin{cases} 0, & x \leq T, \\ m \frac{x - T}{m - T}, & x > T, \end{cases} \quad (5)$$

for  $0 < T < m$ . Thus the background is removed and the surviving range is stretched to the original maximum. The PSF floor is specified as a fraction  $q$  of the source PSF peak,  $T_p = qp_{\max}$ .

### 3.3 Rectangular PSF selection and normalization

After thresholding, all full-array samples outside the accepted red rectangle are set to zero. A compact kernel of height  $M_h$  and width  $N_h$  is extracted around the selected center. Missing samples are zero-padded when the rectangle extends beyond the source array. The calculation PSF is

$$h[u, v] \leftarrow \frac{\max(h_{\text{crop}}[u, v], 0)}{\sum_{u, v} \max(h_{\text{crop}}[u, v], 0)}, \quad (6)$$

so that  $h \geq 0$  and  $\sum h = 1$ .

Pending slider and rectangle changes do not alter the histogram or calculation data until **Apply thresholds / PSF selection now** is pressed. The rectangle can be moved, resized by edges or corners, or scaled about its center with the mouse wheel.

### 3.4 Automatic initial support

The initial support suggestion estimates the border median  $b$  and a robust noise scale

$$\sigma_b \approx 1.4826 \text{ median}(|p_{\partial} - b|). \quad (7)$$

Pixels are active when

$$p[u, v] > \max(b + 3\sigma_b, 10^{-2}p_{\max}). \quad (8)$$

Width and height are estimated independently, which is useful for motion blur or astigmatic PSFs. The proposal remains editable.

## 4 Graphical workflow

### 4.1 Tab 1: image and PSF

The main actions are ordered as: **Load image**, **Load PSF**, **Generate test image**, **Generate selected PSF**, and **Generate degraded input**. Previews, histograms, and resolution labels describe the committed calculation data. If source image and PSF arrays have different dimensions, the smaller array is centered on a common zero-padded canvas; no resampling is performed by this synchronization.

### 4.2 Tab 2: thresholds and calculation PSF

Two 256-bin histograms are located above the corresponding threshold controls. In full-array PSF preview, the area outside the committed rectangle is black because it is numerically zero. The compact preview shows the cropped, normalized kernel.

**Optimize PSF floor + Wiener K** searches for a plausible PSF floor and Wiener regularization. With a reference image it minimizes reconstruction MSE. Without a reference, the floor is constrained by robust PSF-background statistics and GCV selects  $K$  conditionally for each fixed PSF candidate. Kernels that collapse to a few samples or lose nearly all mass are rejected or penalized.

### 4.3 Tab 3: algorithm

Tab 3 selects the reconstruction method and parameters. Optional Wiener initialization, TV processing, and neural denoising are explicit. Auto tuning freezes their enabled/disabled state at the start, so parameters of disabled stages are not changed. Blind algorithms take the PSF width and height directly from the rectangle in Tab 2.

### 4.4 Tab 4: results

All stored iterations are assessed in batches, using Torch/CUDA where practical. After reconstruction the best iteration is selected and **Auto levels** is applied. Black and white controls span the full normalized display interval, maintain a positive gap, and alter visualization only. Cached histograms provide approximate percentiles without repeated full-image sorting.

## 5 Wiener filtering

Every Wiener stage uses explicit FFT and inverse FFT operations. For scalar  $K$  and optional normalized noise spectrum  $N[k, l]$ ,

$$\hat{F}[k, l] = \frac{H^*[k, l]G[k, l]}{|H[k, l]|^2 + KN[k, l]}, \quad (9)$$

followed by

$$\hat{f} = \text{Re}\{\mathcal{F}^{-1}[\hat{F}]\}. \quad (10)$$

The obsolete magnitude-of-IFFT output was removed because it is nonlinear and converts negative ringing into positive intensity.

For fixed PSF, generalized cross-validation uses the leverage

$$L_H = \frac{|H|^2}{|H|^2 + KN}, \quad R_H = 1 - L_H, \quad (11)$$

and a normalized cost of the form

$$\text{GCV}(K) = \frac{\langle |R_H G_c|^2 \rangle / \langle |G_c|^2 \rangle}{\langle R_H \rangle^2}, \quad (12)$$

where  $G_c$  is the spectrum of the mean-centered measured image. GCV selects  $K$  for a fixed PSF; it is not used as unrestricted proof that a strongly thresholded PSF is physically correct.

## 6 Richardson–Lucy family

### 6.1 Classical Richardson–Lucy

The multiplicative update is

$$f^{(t+1)} = f^{(t)} \odot \mathcal{H}_{\text{lin}}^* \left[ \frac{g}{\mathcal{H}_{\text{lin}} f^{(t)} + \varepsilon} \right]. \quad (13)$$

The implementation uses the exact adjoint of the zero-boundary linear *same* operator. Initialization can be a constant image or a Wiener result. Nonnegativity, TV processing, and a neural denoiser can be applied.

### 6.2 Richardson–Lucy–Wiener

This experimental hybrid performs an RL update and then a Wiener operation in every iteration. It combines multiplicative data correction with frequency-domain regularization controlled by  $K$ .

### 6.3 Richardson–Lucy–Rosen

The Rosen variant replaces ordinary back-projection by nonlinear spectral correlation

$$C_{L,M} = (|F_1| + \varepsilon)^L (|F_2| + \varepsilon)^M \exp[i(\arg F_1 - \arg F_2)]. \quad (14)$$

Smaller exponents emphasize phase. An optional relaxation moves  $L$  and  $M$  toward one during the iteration.

## 7 Landweber, Kaczmarz, and Adam methods

### 7.1 Landweber

The classical update is gradient descent on the least-squares data term:

$$f^{(t+1)} = f^{(t)} + \alpha \mathcal{H}_{\text{lin}}^* (g - \mathcal{H}_{\text{lin}} f^{(t)}). \quad (15)$$

The Wiener-preconditioned version transforms the adjoint residual  $d$  as

$$\tilde{d} = \text{Re} \left\{ \mathcal{F}^{-1} \left[ \frac{\mathcal{F}\{d\}}{|H|^2 + KN} \right] \right\} \quad (16)$$

and applies  $f^{(t+1)} = f^{(t)} + \alpha \tilde{d}$ .

### 7.2 Block Kaczmarz

The block Kaczmarz implementation is an experimental ART-style method. Rectangular observation blocks are visited, local residuals are back-projected, and relaxed corrections are applied to the corresponding reconstruction regions. It avoids explicit construction of the very large convolution matrix.

### 7.3 PyTorch Adam TV-MAP

The non-blind Adam method minimizes

$$J(f) = \frac{1}{2} \|\mathcal{H}_{\text{lin}} f - g\|_2^2 + \lambda_f \text{TV}(f) \quad (17)$$

with automatic differentiation. Candidate parameter sets can be processed in batches. Nonnegativity is enforced by projection when selected.

## 8 Blind deconvolution

### 8.1 Blind Richardson–Lucy

Blind RL alternates the image update with a PSF update obtained from correlation of the current image estimate and the relative-blur ratio. The PSF is cropped to the Tab-2 dimensions, projected to nonnegative values, and normalized after each update. Initialization can use the current calculation PSF or a rectangular Gaussian. Optional rotational projection imposes symmetry.

### 8.2 Blind Adam TV-MAP

The blind PyTorch method jointly minimizes

$$J(f, h) = \frac{1}{2} \|h * f - g\|_2^2 + \lambda_f \text{TV}(f) + \lambda_h \text{TV}(h). \quad (18)$$

After every Adam step the PSF is projected onto  $h \geq 0$  and  $\sum h = 1$ , with optional rotational symmetry. Separate learning rates are used for image and PSF.

Blind deconvolution is intrinsically non-unique. PSF width, image sharpness, noise, regularization, initialization, and stopping iteration can compensate for one another.

## 9 Metrics and best-iteration selection

With an independent reference, the program evaluates MSE, PSNR, and SSIM. For normalized peak value one,

$$\text{PSNR} = 10 \log_{10}(1/\text{MSE}). \quad (19)$$

Without a reference it uses a composite no-reference criterion including reblur consistency, intensity preservation, total variation, and residual whiteness. For candidate  $\hat{f}$  the residual is

$$r = \mathcal{H}_{\text{lin}} \hat{f} - g. \quad (20)$$

Residual whiteness is estimated from short-range normalized autocorrelations of  $r - \bar{r}$ . Metrics for the history are computed in batches and cached, so returning to the best iteration does not rerun postprocessing.

## 10 Auto tuning and cancellation

Auto evaluates parameter candidates, using batched implementations when possible. Parameters belonging to optional stages that were disabled when tuning started are frozen. Auto calculations run in an isolated numerical process. **Cancel Auto** first requests cooperative stopping; if the current numerical iteration does not return within five seconds, the worker process is terminated while the Qt GUI process remains alive.

## 11 Batch processing and CUDA

PyTorch variants operate on stacks of candidates where possible. The key choices are: default `float32`; cached spectra for fixed PSFs; differentiable FFT convolution for changing PSFs; memory-limited chunking; grouping of blind-history frames by PSF shape; and CPU fallback when CUDA is unavailable or memory allocation fails. GPU acceleration is most useful for large images, many iterations, and candidate batches.

## 12 Software architecture

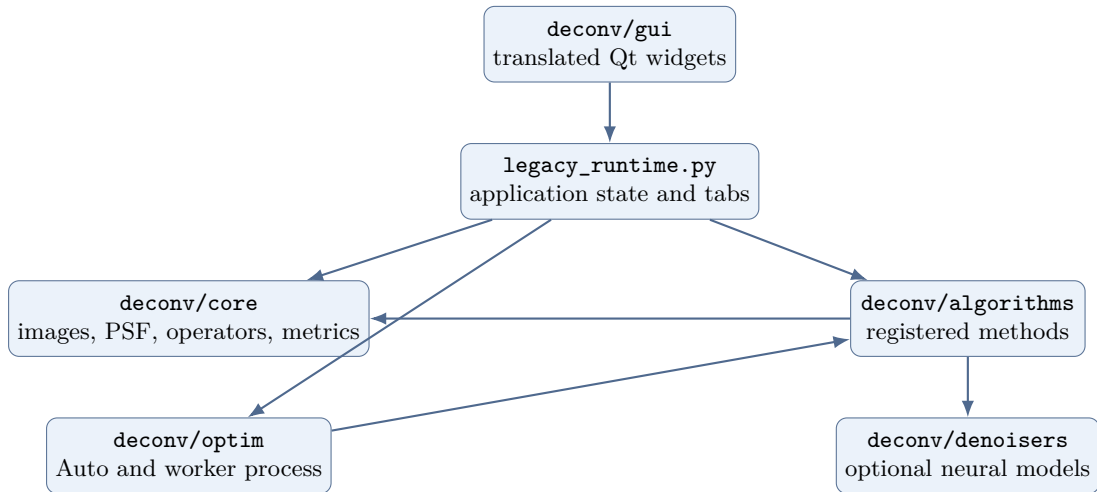


Figure 1: Main project modules and dependencies.

The GUI supports exactly English and Polish. English source strings are stable identifiers. Translated widget wrappers preserve canonical English algorithm names and configuration values. The selected language is stored in the active JSON profile; source comments, docstrings, and numerical identifiers remain English.

The default settings profile is stored in the user configuration directory. On Linux this is normally `~/.config/dekonwolucje`, unless `XDG_CONFIG_HOME` is set. The last profile and image directory are remembered.



## 13 Algorithm registry

| GUI identifier              | Role                                              |
|-----------------------------|---------------------------------------------------|
| Wiener                      | Closed-form circular FFT/IFFT inverse.            |
| Richardson-Lucy             | Multiplicative RL with exact linear-same adjoint. |
| Richardson-Lucy-Wiener      | RL followed by Wiener filtering each iteration.   |
| Richardson-Lucy-Rosen       | RL-like nonlinear spectral-correlation variant.   |
| Blind Richardson-Lucy       | Alternating image and PSF updates.                |
| Landweber                   | Least-squares gradient descent.                   |
| Landweber                   | Landweber with spectral gradient preconditioning. |
| Wiener-preconditioned       |                                                   |
| Block Kaczmarz              | Experimental block/ART residual projections.      |
| Torch batch Wiener          | Batched PyTorch/CUDA Wiener inverse.              |
| Torch batch Richardson-Lucy | Batched RL.                                       |
| Torch batch                 | Batched hybrid RL/Wiener.                         |
| Richardson-Lucy-Wiener      |                                                   |
| Torch batch                 | Batched Rosen variant.                            |
| Richardson-Lucy-Rosen       |                                                   |
| Torch batch Landweber       | Batched Landweber.                                |
| PyTorch Adam TV-MAP         | Non-blind differentiable MAP optimization.        |
| PyTorch Blind Adam TV-MAP   | Joint image/PSF MAP optimization.                 |

## 14 Installation, testing, and extension

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
dekonwolucje
```

Alternative launch commands are `python -m deconv` and `python run_deconvolution_gui.py`. Optional PyTorch support can be installed with `python -m pip install -e ".[gpu]"`; a CUDA-compatible PyTorch wheel must match the local driver environment.

Tests are run with

```
python -m pip install -e ".[test]"
python -m pytest
```

To add a method, derive from the common algorithm interface, use shared data and operators, register the method, add canonical English controls and Polish translations, and add numerical and smoke tests.

## 15 Limitations and interpretation

- Wiener uses a circular model, while most iterative methods use zero-boundary linear convolution.
- Joint PSF-floor and  $K$  selection without a reference is heuristic; GCV selects regularization for a fixed PSF but does not uniquely identify the physical PSF.
- Blind methods are non-convex and depend on support, initialization, constraints, and stopping.
- Display levels alter visualization only, not stored reconstruction values.
- Fine reconstructed structure should be validated against independent measurements or simulations.

## 16 License and project information

The project is distributed under the MIT License.

**Homepage:** <https://github.com/rkotynski/dekonwolucje>